

Day 4 — Aggregate Functions

Today we move from **row-level queries** to **summary-level queries**.

So far:

```
SELECT title  
FROM documents;
```

returns information about individual documents.

Today you'll learn how to ask questions like:

How many documents do we have?

What is the largest document?

What is the average document size?

How many pages are stored across all documents?

These are fundamental for **analytics, monitoring dashboards, RAG ingestion pipelines, and production systems**.

Important: You must write the SQL yourself. I will not give answers before your attempt.

1. Today's Goals

By the end of Day 4, you should understand:

- Aggregate functions
 - COUNT()
 - SUM()
 - AVG()
 - MIN()
 - MAX()
 - COUNT(*)
 - COUNT(column)
 - How NULL affects aggregates
 - Aggregates with WHERE
 - Combining aggregates with aliases
 - Common aggregate mistakes
 - Basic RAG analytics
-

2. Our Database

We'll continue using:

documents

id	user_id	title	file_type	page_count	status
101	1	RAG Fundamentals	pdf	20	processed
102	1	PostgreSQL Guide	pdf	45	processed
103	2	Python Basics	pdf	30	processing
104	3	FastAPI Tutorial	md	15	failed
105	4	Vector Databases	pdf	50	processed
106	5	Docker Guide	pdf	25	processing

We'll also use:

users

id	name	country	account_status
1	Aarav	India	active
2	Emma	USA	active
3	Liam	UK	inactive
4	Sofia	India	active
5	Noah	USA	suspended

3. What Is an Aggregate Function?

An aggregate function takes **multiple rows** and produces a summary value.

For example:

```
SELECT COUNT(*)
FROM documents;
```

Instead of returning:

101
102
103
104
105
106

it returns one summary value:

6

Think:

Many rows

↓

Aggregate function

↓

One summary result

4. COUNT()

COUNT() is probably the aggregate function you'll use most.

COUNT(*)

```
SELECT COUNT(*)  
FROM documents;
```

Meaning:

Count the rows in documents.

For our dataset:

6

5. COUNT(column)

You can also write:

```
SELECT COUNT(title)  
FROM documents;
```

This counts rows where title is **not NULL**.

This distinction is extremely important.

Suppose the table contained:

id	title
1	RAG
2	PostgreSQL
3	NULL
4	FastAPI

Then:

COUNT(*)

would be:

4

while:

COUNT(title)

would be:

3

because one title is NULL.

6. COUNT(*) vs COUNT(column)

Memorize this:

COUNT(*) → counts rows

COUNT(column) → counts non-NULL values in that column

This distinction is frequently tested in SQL interviews.

7. Give Aggregate Results a Name

This:

```
SELECT COUNT(*)  
FROM documents;
```

works, but the result column may have an ugly generated name.

Use:

```
SELECT COUNT(*) AS total_documents  
FROM documents;
```

Result:

```
total_documents
```

```
6
```

Aliases make SQL results much easier for applications to consume.

For example, your FastAPI backend can return:

```
{  
  "total_documents": 6  
}
```

8. SUM()

SUM() adds numeric values.

For example:

```
SELECT SUM(page_count)  
FROM documents;
```

Our pages are:

20

45

30

15

50

25

Conceptually:

$20 + 45 + 30 + 15 + 50 + 25$

Result:

185

Better:

```
SELECT SUM(page_count) AS total_pages
FROM documents;
```

9. RAG Example — Total Pages

Suppose your document-processing team wants:

"How many pages has our system ingested?"

You could use:

```
SELECT
SUM(page_count) AS total_pages
FROM documents;
```

This could be used in an ingestion dashboard.

10. AVG()

AVG() calculates an average.

```
SELECT AVG(page_count)
FROM documents;
```

Conceptually:

total pages

number documents

For our dataset:

185 / 6

≈

30.83

Use an alias:

```
SELECT
AVG(page_count) AS average_pages
FROM documents;
```

11. PostgreSQL Numeric Types

PostgreSQL may return an average using a numeric type rather than an integer.

That's useful because:

30.83

contains decimal precision.

Don't assume aggregate results always have the same type as the source column.

We'll explore explicit type casting later.

12. MIN()

Find the smallest value:

```
SELECT
```

```
MIN(page_count) AS smallest_document  
FROM documents;
```

Result:

15

Business interpretation:

The smallest document contains 15 pages.

13. MAX()

Find the largest value:

```
SELECT
```

```
MAX(page_count) AS largest_document  
FROM documents;
```

Result:

50

Business interpretation:

The largest document contains 50 pages.

14. Five Core Aggregate Functions

You should memorize these:

Function	Purpose
COUNT()	Count
SUM()	Total
AVG()	Average
MIN()	Minimum
MAX()	Maximum

Example:

```
SELECT
```

```
COUNT(*) AS total_documents,
```

```
SUM(page_count) AS total_pages,
```

```
AVG(page_count) AS average_pages,
```

```
MIN(page_count) AS minimum_pages,
```

MAX(page_count) AS maximum_pages

FROM documents;

This returns **one row** containing several metrics.

15. One Query — Multiple Metrics

This is extremely useful in production.

Suppose your dashboard needs:

Total documents

Total pages

Average pages

Smallest document

Largest document

Instead of making five database queries:

Query 1 → COUNT

Query 2 → SUM

Query 3 → AVG

Query 4 → MIN

Query 5 → MAX

you can often calculate them together:

SELECT

COUNT(*) AS total_documents,

SUM(page_count) AS total_pages,

AVG(page_count) AS average_pages,

MIN(page_count) AS minimum_pages,

MAX(page_count) AS maximum_pages

FROM documents;

One database request.

This is a very common analytics pattern.

16. Aggregates + WHERE

Aggregates become much more useful when combined with filtering.

Suppose you want:

Number of processed documents.

SELECT

COUNT(*) AS processed_documents

```
FROM documents
```

```
WHERE status = 'processed';
```

Now WHERE filters the rows **before** the aggregate calculates the result.

Mental model:

All documents

↓

```
WHERE status = processed
```

↓

```
COUNT(*)
```

↓

result

This is very important.

17. Another Example

Question:

What is the total number of pages in processed documents?

You need:

filter → aggregate

Conceptually:

```
SELECT
```

```
SUM(page_count) AS processed_pages
```

```
FROM documents
```

```
WHERE status = 'processed';
```

18. Aggregate + Multiple Conditions

Question:

What is the average page count of processed PDF documents?

You need:

```
SELECT
```

```
AVG(page_count) AS average_pdf_pages
```

```
FROM documents
```

```
WHERE file_type = 'pdf'
```

```
AND status = 'processed';
```

Notice the pattern:

```
FROM
```

WHERE

SELECT aggregate

Don't worry about the fact that SELECT appears first syntactically. Conceptually, filtering must happen before the aggregate can summarize the filtered rows.

19. NULL and Aggregates

This is another important concept.

Suppose:

id	page_count
1	20
2	40
3	NULL

Then:

```
SELECT SUM(page_count)
```

```
FROM documents;
```

ignores the NULL value.

Similarly, AVG(), MIN(), and MAX() generally ignore NULL inputs.

For counting:

```
COUNT(page_count)
```

counts only non-NULL page_count values.

But:

```
COUNT(*)
```

counts rows regardless of whether page_count is NULL.

20. Important NULL Example

Imagine:

100 documents

but:

10 documents have page_count = NULL

Then:

```
COUNT(*)
```

returns:

100

while:

```
COUNT(page_count)
```

returns:

This can be useful when asking:

How many documents have a known page count?

21. Common Mistake — Mixing Aggregate and Normal Columns

Consider:

```
SELECT
```

```
title,
```

```
COUNT(*)
```

```
FROM documents;
```

You may expect:

```
title | count
```

But SQL has a problem:

Which title should represent all six rows?

There are six different titles.

We'll solve this properly with GROUP BY on **Day 5**.

For today, understand the core principle:

An aggregate query that produces one summary row cannot arbitrarily select an ordinary row-level column alongside it.

22. Another Common Mistake

Developer writes:

```
SELECT
```

```
COUNT(*) AS total_documents,
```

```
title
```

```
FROM documents;
```

Ask yourself:

One count summarizes all rows. Which title should PostgreSQL return?

There isn't one obvious answer.

That's why grouping exists.

23. COUNT(*) Is Usually Your Default Row Count

When your requirement is:

How many rows are there?

Prefer:

COUNT(*)

rather than:

COUNT(id)

assuming you simply want row count.

Why?

Because COUNT(*) explicitly communicates:

Count rows.

We'll discuss performance and PostgreSQL internals later.

24. RAG Monitoring Example

Suppose your ingestion pipeline has:

processed

processing

failed

You might want:

How many documents have failed?

SELECT

COUNT(*) AS failed_documents

FROM documents

WHERE status = 'failed';

Or:

How many PDF documents are currently processing?

SELECT

COUNT(*) AS processing_pdfs

FROM documents

WHERE file_type = 'pdf'

AND status = 'processing';

These are realistic production queries.

25. Debugging Challenge #1

A developer writes:

SELECT

title,

COUNT(*) AS total_documents

FROM documents;

PostgreSQL rejects it.

Your task:

1. Why is this invalid?
 2. What concept are we missing?
 3. Don't fix it using GROUP BY yet—we haven't learned it.
 4. Explain what PostgreSQL doesn't know.
-

26. Debugging Challenge #2

Developer wants:

Count documents whose page_count is missing.

They write:

```
SELECT COUNT(page_count)
FROM documents
WHERE page_count = NULL;
```

Tasks:

1. What's wrong?
 2. Correct the query.
 3. Explain both the COUNT() and NULL behavior.
-

27. Exercise Set A — Basic Aggregates

Using documents:

Q1

Count all documents.

Return the result as:

total_documents

Q2

Calculate the total number of pages.

Return:

total_pages

Q3

Calculate the average page count.

Return:

average_pages

Q4

Find the smallest document by page count.

Return:

minimum_pages

Q5

Find the largest document by page count.

Return:

maximum_pages

28. Exercise Set B — Multiple Aggregates

Q6

Write **one query** that returns:

total_documents

total_pages

average_pages

minimum_pages

maximum_pages

Q7

Count only processed documents.

Return:

processed_documents

Q8

Calculate the total pages of processed documents.

Return:

processed_pages

Q9

Calculate the average page count of PDF documents.

Return:

average_pdf_pages

Q10

Calculate the maximum page count among failed documents.

Return:

largest_failed_document

29. Exercise Set C — COUNT(*) vs COUNT(column)

Imagine another table:

document_processing

id	document_id	started_at	completed_at
1	101	10:00	10:05
2	102	10:10	10:20
3	103	10:30	NULL
4	104	11:00	11:05

Q11

What does this return?

```
SELECT COUNT(*)  
FROM document_processing;
```

Don't just give the number. Explain.

Q12

What does this return?

```
SELECT COUNT(completed_at)  
FROM document_processing;
```

Explain why it differs from Q11.

Q13

Write SQL to count how many processing jobs have **not completed yet**.

30. Practical Engineering Problem

Your RAG monitoring dashboard needs a single database query returning:

total_documents

processed_documents

failed_documents

processing_documents

total_pages

average_pages

Use the documents table.

Requirements

- One query
- Appropriate aggregate functions
- Clear aliases
- No GROUP BY yet
- Don't manually calculate the numbers

This should be a **single summary row**.

31. Practical RAG Problem

Your team wants to understand the current ingestion workload.

Requirement:

Find the total number of pages across documents that are currently processing, and the average number of pages per processing document.

Return:

processing_pages

average_processing_pages

Write one query.

32. Production Debugging

A developer writes:

```
SELECT
```

```
COUNT(*) AS total_documents,
```

```
SUM(page_count) AS total_pages
```

```
FROM documents
```

```
WHERE status = 'processed'
```

```
OR status = 'processing'
```

AND file_type = 'pdf';

The requirement is:

Count/sum only PDF documents whose status is either processed or processing.

Your tasks:

1. Identify the logical problem.
2. Explain operator precedence.
3. Rewrite the query correctly.
4. Explain why parentheses are important.

This combines **Day 2 + Day 4**.

33. Interview Questions

Answer these in your own words.

Q1

What is an aggregate function?

Q2

What's the difference between:

COUNT(*)

and:

COUNT(column_name)

Q3

How does NULL affect COUNT(column_name)?

Q4

What do these do?

SUM()

AVG()

MIN()

MAX()

Q5

Why is this problematic?

```
SELECT title, COUNT(*)
```

```
FROM documents;
```

Don't just say "syntax error."

Explain the **logical problem**.

34. Explanation Challenge

Explain this query as if you're explaining it to a backend engineer:

```
SELECT
```

```
COUNT(*) AS total_documents,
```

```
SUM(page_count) AS total_pages,
```

```
AVG(page_count) AS average_pages,
```

```
MIN(page_count) AS smallest_document,
```

```
MAX(page_count) AS largest_document
```

```
FROM documents
```

```
WHERE status = 'processed';
```

Your explanation should answer:

1. What rows are considered?
 2. What happens to those rows?
 3. How many rows does the query return?
 4. What does each output column represent?
-

35. Data Engineering Thinking

Suppose your RAG system contains:

10 million documents

You run:

```
SELECT
```

```
COUNT(*),
```

```
SUM(page_count),
```

```
AVG(page_count)
```

```
FROM documents;
```

Answer conceptually:

Is this query returning 10 million rows?

Why or why not?

And:

Why can aggregate queries still be expensive even though they return only one row?

Think about how much data PostgreSQL may need to examine.

36. Day 4 Short Test

Q1

What does COUNT(*) count?

Q2

What does COUNT(page_count) count?

Q3

Write a query to count all documents.

Q4

Write a query to calculate total pages.

Q5

Write a query to calculate average pages.

Q6

Write a query to find the maximum page count.

Q7

What happens to NULL values in AVG(page_count)?

Q8

Why is this problematic?

```
SELECT title, COUNT(*)  
FROM documents;
```

Q9

Write a query to count failed documents.

Q10

Write one query returning:

total_documents

total_pages

average_pages

for only processed documents.
